

Chapter-8

SOFTWARE TESTING

Software Testing

- Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use.
- When you test software, you execute a program using artificial data.
- You check the results of the test run for errors, anomalies or information about the program's non-functional attributes.

Testing process goals

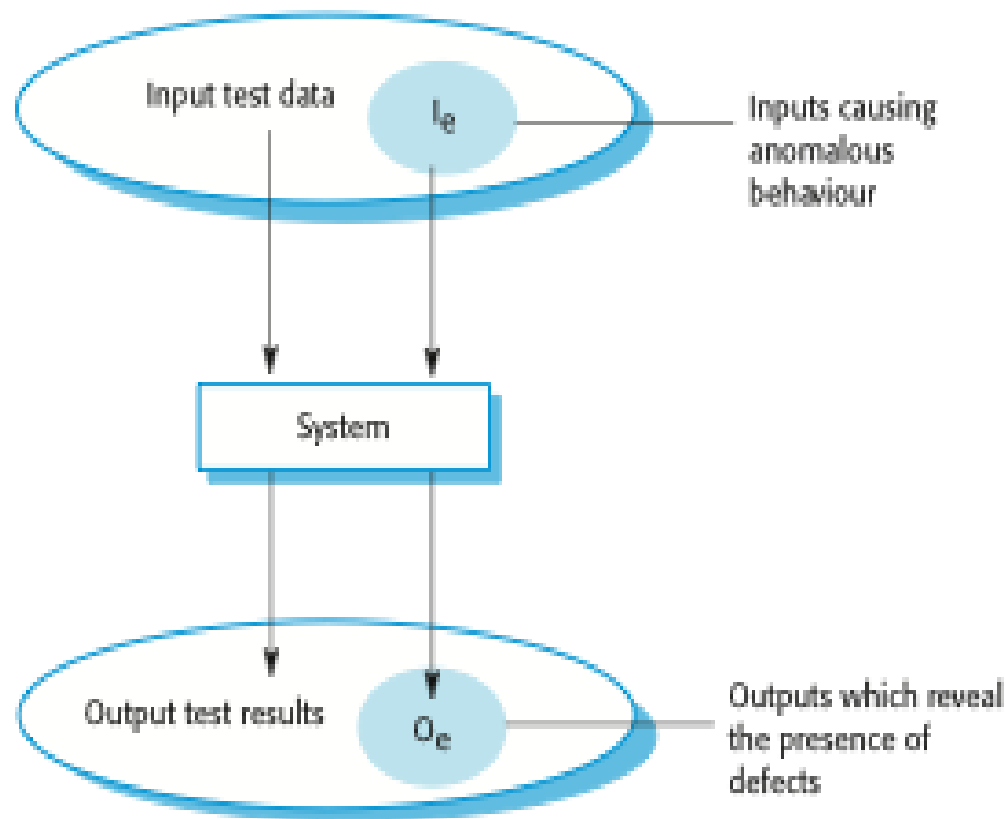
➤ Validation testing

- To demonstrate to the developer and the system customer that the software meets its requirements
- A successful test shows that the system operates as intended.

➤ Defect testing

- To discover faults or defects in the software where its behavior is incorrect or not in conformance with its specification
- A successful test is a test that makes the system perform incorrectly and so exposes a defect in the system.

An input-output model of program testing



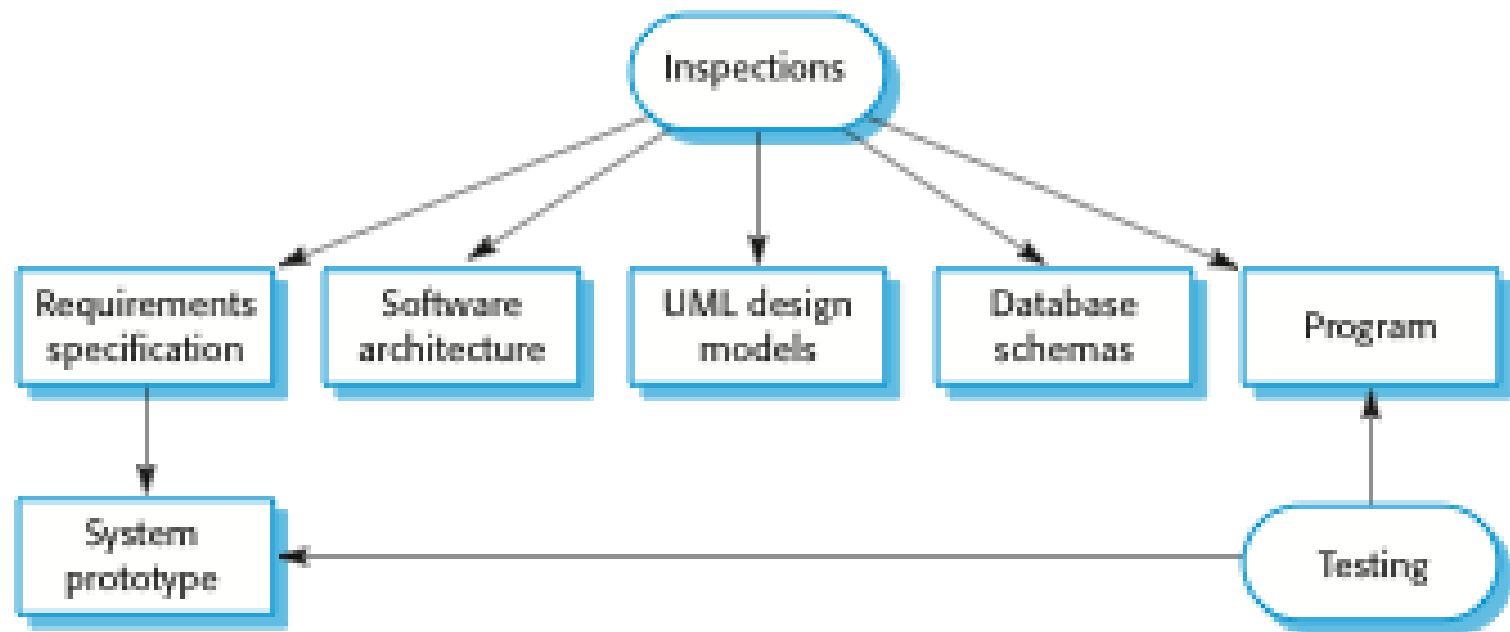
Inspections and testing

- **Software inspections** Concerned with analysis of the static system representation to discover problems (static verification)
- **Software testing** Concerned with exercising and observing product behaviour (dynamic verification)

Software inspections

- These involve people examining the source representation with the aim of discovering anomalies and defects.
- Inspections not require execution of a system so may be used before implementation.
- They may be applied to any representation of the system (requirements, design, configuration data, test data, etc.).
- They have been shown to be an effective technique for discovering program errors.

Software inspections



Stages of testing

- Development testing, where the system is tested during development to discover bugs and defects.
- Release testing, where a separate testing team test a complete version of the system before it is released to users.
- User testing, where users or potential users of a system test the system in their own environment.

Development Testing

- Development testing includes all testing activities that are carried out by the team developing the system.
 - **Unit testing**, where individual program units or object classes are tested. Unit testing should focus on testing the functionality of objects or methods.
 - **Component testing**, where several individual units are integrated to create composite components. Component testing should focus on testing component interfaces.
 - **System testing**, where some or all of the components in a system are integrated and the system is tested as a whole. System testing should focus on testing component interactions.

Unit testing

- Unit testing is the process of testing individual components in isolation.
- Units may be:
 - Individual functions or methods within an object
 - Object classes with several attributes and methods
- Tests should be call to these methods with different input parameters
- Complete test coverage of a class involves
 - Testing all operations associated with an object
 - Setting and interrogating all object attributes
 - Exercising the object in all possible states.

The weather station object interface

WeatherStation
identifier
reportWeather () reportStatus () powerSave (instruments) remoteControl (commands) reconfigure (commands) restart (instruments) shutdown (instruments)

Weather station testing

- Need to define test cases for reportWeather, test, startup and shutdown.
- Using a state model, identify sequences of state transitions to be tested and the event sequences to cause these transitions
- For example:
 - Shutdown -> Running-> Shutdown
 - Configuring-> Running-> Testing -> Transmitting -> Running
 - Running-> Collecting-> Running-> Summarizing -> Transmitting -> Running

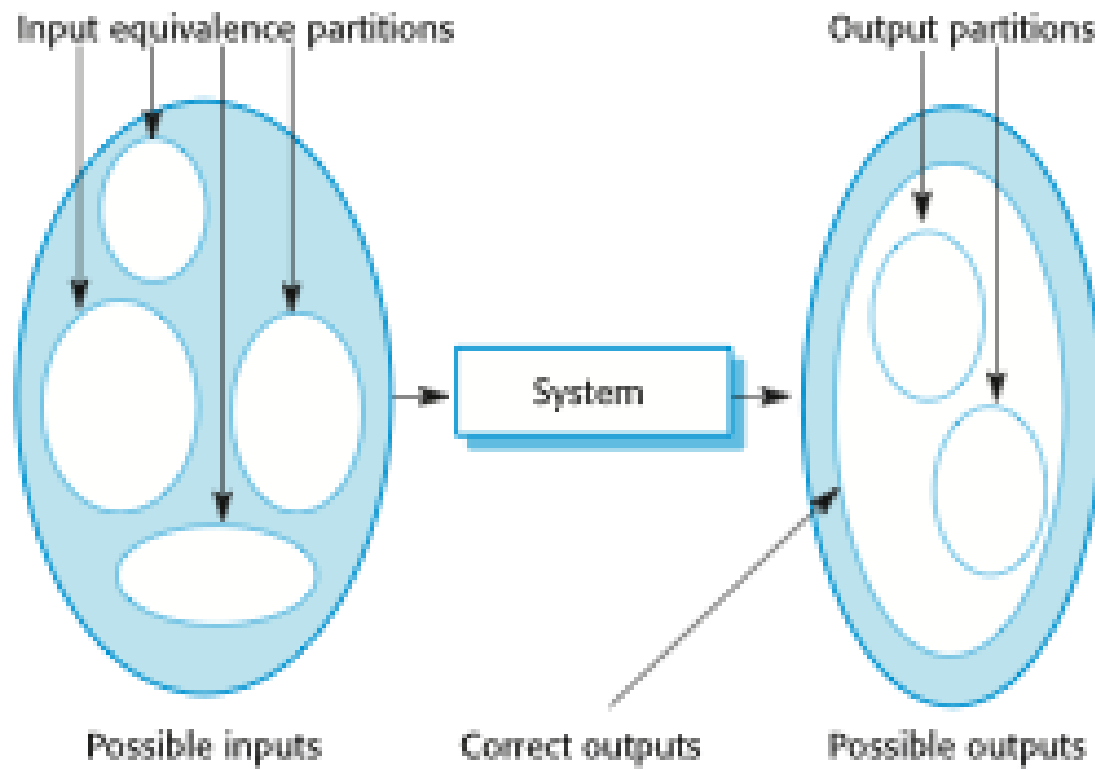
Unit test effectiveness

- This leads to 2 types of unit test case:
 - The first of these should reflect normal operation of a program and should show that the component works as expected.
 - The other kind of test case should be based on testing experience of where common problems arise. It should use abnormal inputs to check that these are properly processed and do not crash the component.

Partition testing

- Input data and output results often fall into different classes where all members of a class are related.
- Each of these classes is an **equivalence partition** or domain where the program behaves in an equivalent way for each class member.
- Test cases should be chosen from each partition.

Partition testing



Partition testing

- **Equivalence Partitioning** – It is a black box technique (code is not visible to tester) which can be applied to all levels of testing like unit, integration, system etc. In this technique, you divide set of test condition into partition that can be considered the same

Equivalence Partitioning-Example

The screenshot shows a 'Flight Reservation' application window. It has a menu bar with 'File', 'Edit', 'Analysis', and 'Help'. Below the menu is a toolbar with icons for file operations and a help icon. The main area is divided into sections: 'Flight Schedule' with fields for 'Date of Flight' (00/15/01), 'Fly From' (Frankfurt), and 'Fly To' (Denver); 'Order Information' with fields for 'Flight No.', 'Name' (Guru99.com), 'Class' (First), and 'Tickets' (1); and a 'Total' field showing '\$0.00'. A red box highlights the 'Tickets' field, and a speech bubble points to it with the text: 'How system decides, how many reservation users should be allowed?'. The 'Tickets' field is currently set to '1'.

Flight Reservation

File Edit Analysis Help

Flight Schedule:

Date of Flight: 00/15/01 Fly From: Frankfurt Fly To: Denver

Order Information:

Flight No.: Flight No. Time: Airline:

Name: Guru99.com

Class: ☐ First ☐ Business

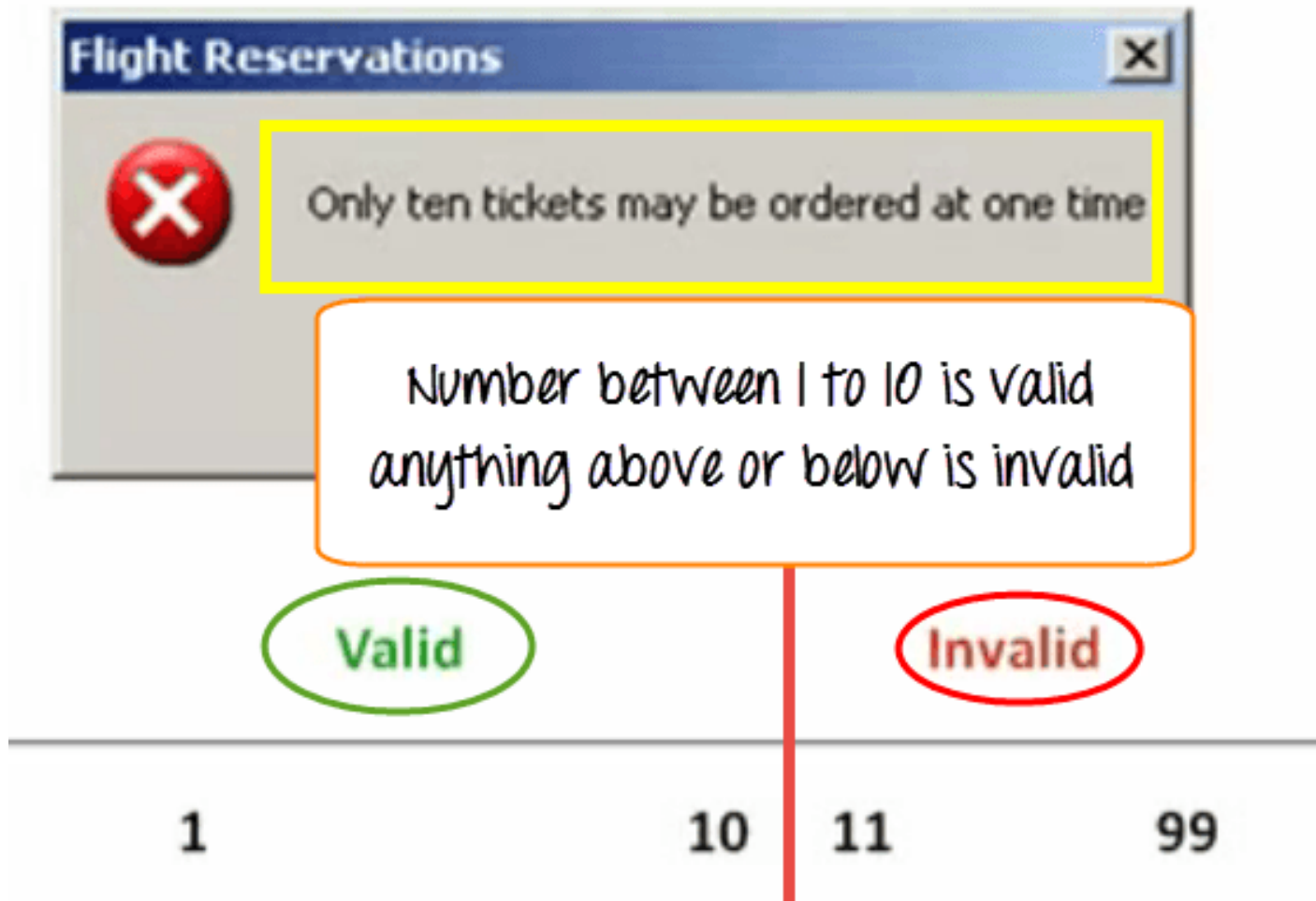
Tickets: 1

Total: \$0.00

Update Order Insert Order

Order No:

Equivalence Partitioning-Example



Equivalence Partitioning-Example

Here is the test condition

- Any Number greater than 10 entered in the reservation column (let say 11) is considered invalid.
- Any Number less than 1 that is 0 or below, then it is considered invalid.
- Numbers 1 to 10 are considered valid
- Any 3 Digit Number say -100 is invalid

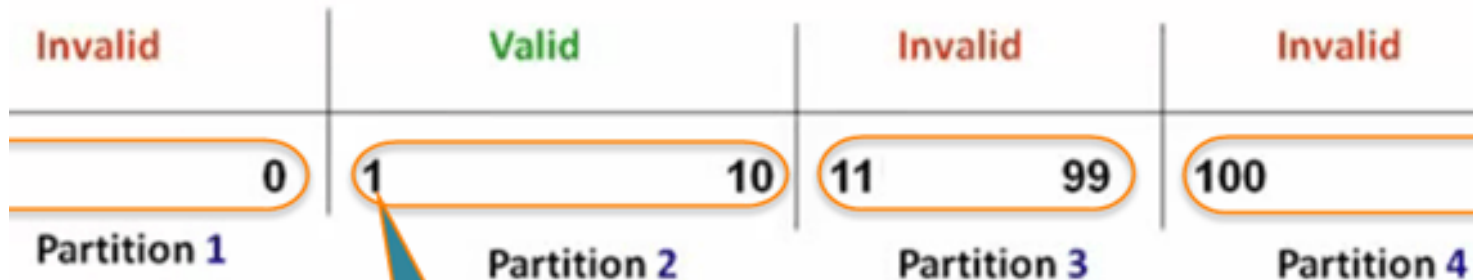
Equivalence Partitioning-Example



The divided sets are called Equivalence Partitions or Equivalence Classes. Then we pick only one value from each partition for testing. The hypothesis behind this technique is ***that if one condition/value in a partition passes all others will also pass***. Likewise, ***if one condition in a partition fails, all other conditions in that partition will fail***.

Boundary Value Analysis

- **Boundary Value Analysis-** in Boundary Value Analysis, you test boundaries between equivalence partitions



IN BOUNDARY VALUE ANALYSIS you will check the boundary values like 0, 1, 10, 11, 99, 100

Black-box and white-box testing

When you use the specification of a system to identify test cases, this is called '**black-box testing**'.

Here, you don't need any knowledge of how the system works.

In '**white-box testing**', you look at the code of the program to find possible test cases.

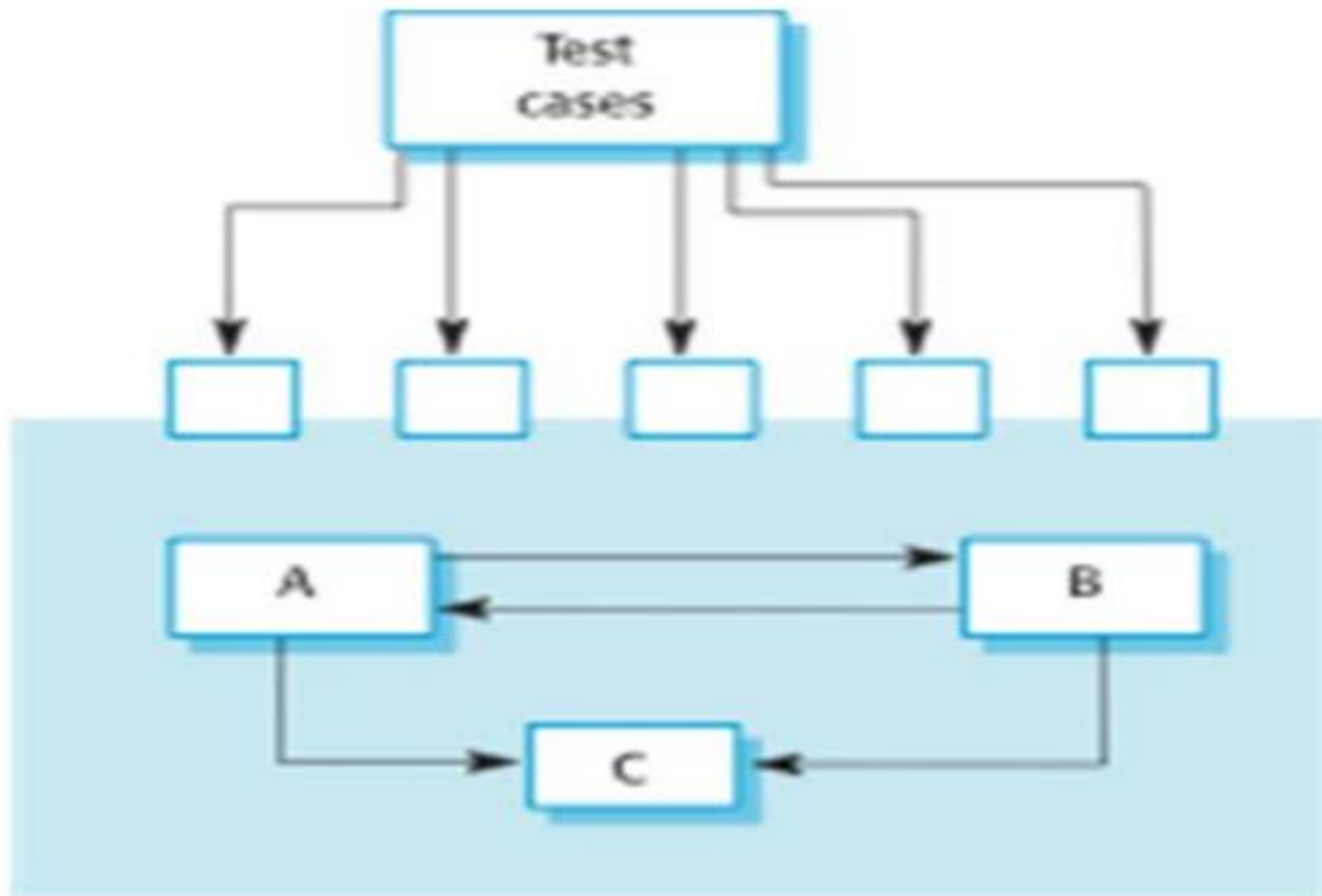
Testing guidelines (sequences)

- Test software with sequences which have only a single value.
- Use sequences of different sizes in different tests.
- Derive tests so that the first, middle and last elements of the sequence are accessed.
- Test with sequences of zero length.

General testing guidelines

- Choose inputs that force the system to generate all error messages
- Design inputs that cause input buffers to overflow
- Repeat the same input or series of inputs numerous times
- Force invalid outputs to be generated
- Force computation results to be too large or too small.

Interface testing



Interface testing

Objectives are to detect faults due to interface errors or invalid assumptions about interfaces.

Interface types

- **Parameter interfaces** Data passed from one method or procedure to another.
- **Shared memory interfaces** Block of memory is shared between procedures or functions.
- **Procedural interfaces** Sub-system encapsulates a set of procedures to be called by other sub-systems
- **Message passing interfaces** These are interfaces in which one component requests a service from another component by passing a message to it.

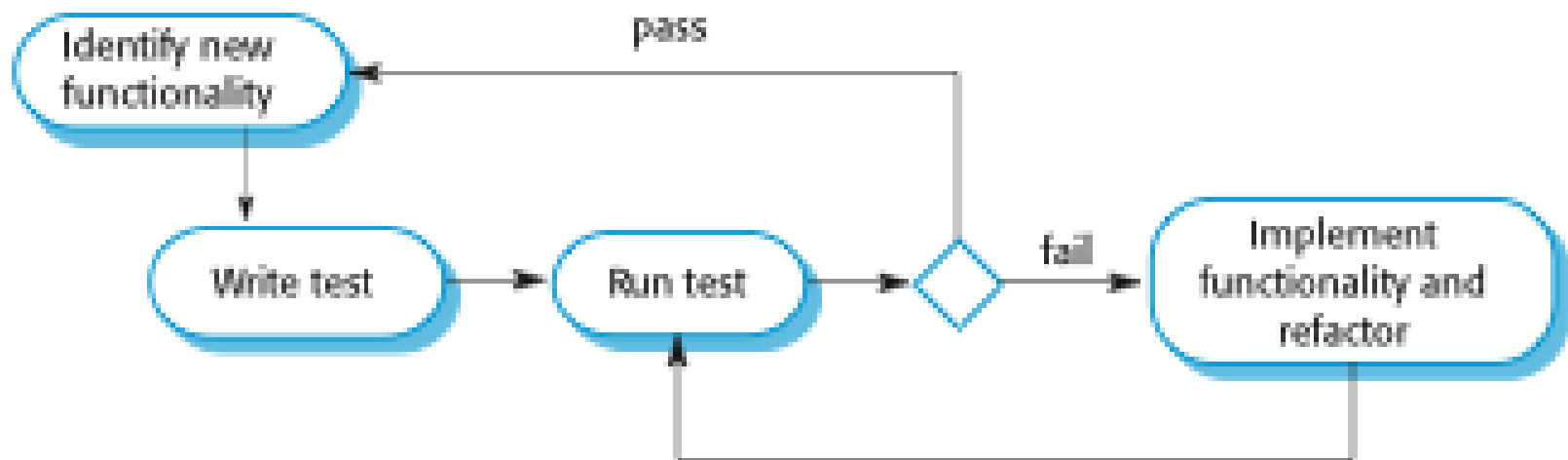
System testing...

- System testing during development involves integrating components to create a version of the system and then testing the integrated system.
- The focus in system testing is testing the interactions between components.
- System testing checks that components are compatible, interact correctly and transfer the right data at the right time across their interfaces.

...System testing

- During system testing, reusable components that have been separately developed and off-the-shelf systems may be integrated with newly developed components. The complete system is then tested.
- Components developed by different team members or sub-teams may be integrated at this stage. System testing is a collective rather than an individual process.
 - In some companies, system testing may involve a separate testing team with no involvement from designers and programmers.

Test-driven development



Benefits of test-driven development

- Code coverage
 - Every code segment that you write has at least one associated test so all code written has at least one test.
- Regression testing
 - A regression test suite is developed incrementally as a program is developed.
- Simplified debugging
 - When a test fails, it should be obvious where the problem lies. The newly written code needs to be checked and modified.
- System documentation
 - The tests themselves are a form of documentation that describe what the code should be doing.

Release testing...

- Release testing is the process of testing a particular release of a system that is intended for use outside of the development team.
- The primary goal of the release testing process is to convince the supplier of the system that it is good enough for use.
 - Release testing, therefore, has to show that the system delivers its specified functionality, performance and dependability, and that it does not fail during normal use.
- Release testing is usually a black-box testing process where tests are only derived from the system specification.

...Release testing

- Release testing is a form of system testing.
- Important differences:
 - A separate team that has not been involved in the system development, should be responsible for release testing.
 - System testing by the development team should focus on discovering bugs in the system (defect testing). The objective of release testing is to check that the system meets its requirements and is good enough for external use (validation testing).

Performance testing

- Part of release testing may involve testing the emergent properties of a system, such as performance and reliability.
- Tests should reflect the profile of use of the system.
- Performance tests usually involve planning a series of tests where the load is steadily increased until the system performance becomes unacceptable.
- Stress testing is a form of performance testing where the system is deliberately overloaded to test its failure behavior.

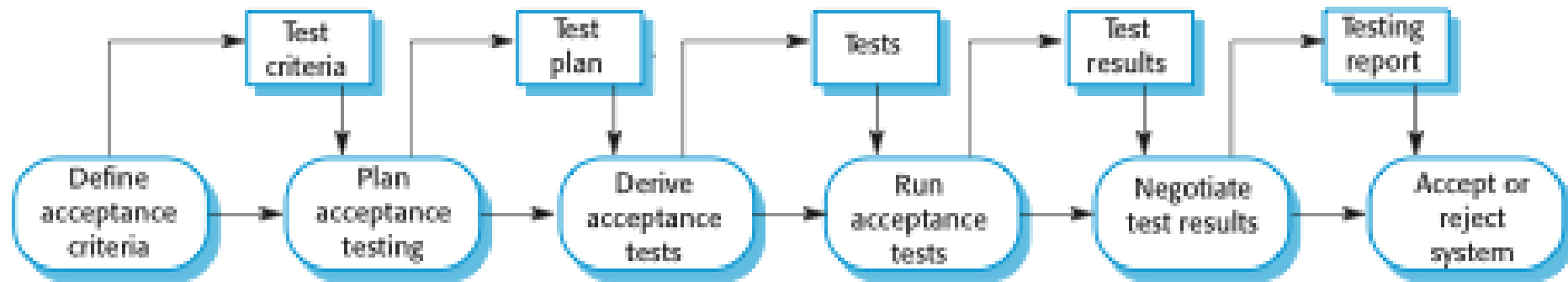
User testing

- User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing.
- User testing is essential, even when comprehensive system and release testing have been carried out.
 - The reason for this is that influences from the user's working environment have a major effect on the reliability, performance, usability and robustness of a system. These cannot be replicated in a testing environment.

Types of user testing

- There are three different types of user testing:
- 1. **Alpha testing**, where a selected group of software users work closely with the development team to test early releases of the software.
- 2. **Beta testing**, where a release of the software is made available to a larger group of users to allow them to experiment and to raise problems that they discover with the system developers.
- 3. **Acceptance testing**, where customers test a system to decide whether or not it is ready to be accepted from the system developers and deployed in the customer environment.

The acceptance testing process



Stages in the acceptance testing process

- Define acceptance criteria
- Plan acceptance testing
- Derive acceptance tests
- Run acceptance tests
- Negotiate test results
- Reject/accept system